

Go-Zero 企业级 IM 项目后续升级计划：前端界面与局域网可用 App 化

一、升级目标

前面 V1-V15 主要解决后端企业级能力，包括可靠消息、多 Gateway 路由、离线补偿、Kafka 群聊扩散、好友群组、会话列表、文件消息、监控和 AI 事件通知。

但要让整个项目达到“完整企业级项目”展示水平，还需要继续升级：

```text id="liauhv" V16：前端 Web 客户端 V17：移动端适配与 PWA 化 V18：局域网部署与一键启动 V19：管理后台界面 V20：监控运维展示页 V21：真实使用场景闭环 V22：桌面端 / App 壳封装 V23：项目演示、文档、压测和简历包装最终版

最终目标是做成：

```text id="6fkdul"  
一个可以在局域网内真实使用的企业级分布式 IM 系统

而不只是后端接口项目。

二、最终系统形态

推荐最终形态

```text id="tbtq14" 后端：Go-Zero 微服务 前端：Vue3 / React Web 客户端 部署：Docker Compose 局域网一键启动 访问方式：电脑浏览器 + 手机浏览器 + PWA 桌面图标 可选扩展：Electron / Tauri / Android WebView 壳

## 局域网使用方式

服务端部署在一台电脑或服务器上：

```text id="8v5wy9"  
局域网服务器 IP：192.168.1.100
前端访问：http://192.168.1.100
后端 API：http://192.168.1.100/api
WebSocket：ws://192.168.1.100/ws

局域网内其他设备：

```text id="e8z5ul" 电脑浏览器打开: http://192.168.1.100 手机浏览器打开: http://192.168.1.100 手机添加到桌面, 形成类 App 使用体验

这样不用上架应用商店, 也不用复杂移动端开发, 就能做到“局域网内可实用”。

---

# 三、推荐技术选型

## 前端技术栈

优先推荐:

```text id="s5smm7"

Vue3 + Vite + TypeScript + Pinia + Vue Router + Element Plus / Naive UI

原因:

```text id="s6d4st" 1. 上手快 2. 页面搭建快 3. 适合后台和 IM Web 界面 4. Codex 生成和维护成本低 5. 可以较容易做响应式适配

如果你更习惯 React, 也可以用:

```text id="n20qq0"

React + Vite + TypeScript + Zustand + Ant Design

但建议只选一个, 不要纠结。

移动端适配

推荐先做:

```text id="o62sfl" 响应式 Web + PWA

不要一开始做 Flutter / React Native / uni-app。

原因:

```text id="tfxyqs"

1. 你现在主线是后端秋招
2. 原生 App 会吃掉大量时间
3. PWA 足够展示“类 App”
4. 局域网使用不需要上架
5. 前端代码可以复用

App 化路线

分三步：

```text id="j63m92" 第一步：手机浏览器可用 第二步：PWA 添加到桌面 第三步：Electron / Tauri / Android WebView 封装

秋招前做到第二步就够。

---

# 四、版本升级规划

# V16：前端 Web 客户端基础版

## 目标

先做一个能真实聊天的 Web 前端，不追求特别好看，但要完整闭环。

## 页面模块

```text id="i9thtk"

1. 登录页
2. 注册页
3. 主布局
4. 会话列表
5. 聊天窗口
6. 好友列表
7. 群组列表
8. 用户设置页

核心能力

1. 登录注册

```text id="snt3wx" 用户注册 用户登录 保存 token 退出登录 token 过期处理

### 2. WebSocket 连接

```text id="4b31bf"

登录后建立 WebSocket
携带 token 鉴权
收到消息后更新聊天窗口
断线后自动重连
重连后触发离线补偿

3. 会话列表

显示：

```
```text id="oiqweg" 头像 会话名 最后一条消息 最后消息时间 未读数
```

### 4. 聊天窗口

支持：

```
```text id="h2den2"
```

发送文本消息

接收实时消息

消息状态：发送中 / 已发送 / 已送达 / 失败

历史消息分页加载

滚动到底部

5. 好友与群组

支持：

```
```text id="i8ml0a" 好友列表 添加好友 创建群聊 群成员列表
```

## 前端目录结构

```
```text id="ycqpmk"
```

web/

src/

api/

auth.ts

user.ts

conversation.ts

message.ts

contact.ts

group.ts

ws/

socket.ts

heartbeat.ts

reconnect.ts

store/

auth.ts

conversation.ts

message.ts

contact.ts

pages/

Login.vue

Register.vue

Chat.vue

Contacts.vue

```

    Groups.vue
    Settings.vue
  components/
    ConversationList.vue
    ChatWindow.vue
    MessageItem.vue
    MessageInput.vue
    UserAvatar.vue
  router/
    index.ts
  utils/
    request.ts
    time.ts
    storage.ts

```

验收标准

```text id="qvg0tr" 1. 用户可以在前端注册登录 2. 登录后可以建立 WebSocket 连接 3. 两个浏览器用户可以实时互发消息 4. 刷新页面后 token 仍然有效 5. WebSocket 断开后可以自动重连 6. 会话列表能展示最后一条消息和未读数 7. 可以分页加载历史消息

## Codex 任务

```text id="ofyeqv"

请为当前 Go-Zero IM 项目新增 Vue3 + Vite + TypeScript 前端客户端。要求实现登录、注册、会话列表、聊天窗口、WebSocket 连接、自动重连、文本消息发送接收、历史消息分页加载和未读数展示。前端通过 REST API 获取会话、好友、群组和历史消息，通过 WebSocket 收发实时消息。请保持前后端接口清晰，生成 web/ 目录，不要修改后端核心逻辑。

V17：移动端适配与 PWA 化

目标

让系统在手机浏览器中可用，并能添加到桌面，形成类 App 体验。

核心能力

```text id="nmp3jp" 响应式布局 移动端聊天界面 PWA manifest Service Worker 添加到桌面 离线提示

## 移动端布局

手机端页面结构：

```text id="d91lm1"

底部 Tab：

1. 消息
2. 联系人
3. 群组
4. 我的

聊天页：

```text id="6d12vp" 顶部：返回 + 会话名 中间：消息列表 底部：输入框 + 发送按钮

## PWA 配置

新增：

```text id="ia09ot"  
manifest.json
service-worker.js
icons/

manifest 示例字段：

```text id="y1k6dn" name short\_name start\_url display: standalone background\_color theme\_color  
icons

## 验收标准

- ```text id="yx4a0l"
1. 手机浏览器访问界面不变形
  2. 可以正常登录和聊天
  3. 可以添加到手机桌面
  4. 从桌面打开后像独立 App
  5. WebSocket 重连机制正常

## Codex 任务

```text id="uy1o5b" 请在现有 Vue3 IM 前端中增加移动端响应式适配和 PWA 支持。要求手机浏览器访问时切换为移动端布局，包含消息、联系人、群组、我的四个 Tab；聊天页适配手机屏幕；新增 manifest.json 和 Service Worker，使系统可以添加到手机桌面并以 standalone 模式运行。

V18：局域网部署与一键启动

目标

让项目可以在局域网内一键启动并真实使用。

部署架构

```
```text id="mytpgt"
```

Nginx

—— /	-> 前端静态文件
—— /api	-> API Gateway
—— /ws	-> WebSocket Gateway

后端服务：

- im-api-gateway
- im-auth
- im-user
- im-gateway
- im-logic
- im-transfer
- im-contact
- im-group
- im-conversation
- im-message

依赖：

- MySQL
- Redis
- Kafka
- Etcd
- MinIO

## Docker Compose

需要支持：

```
```text id="wbdj5d" docker compose up -d
```

启动后：

```
```text id="cfbdt8"
```

前端：http://localhost

局域网：http://本机IP

API：http://本机IP/api

WebSocket：ws://本机IP/ws

MinIO 控制台：http://本机IP:9001

Grafana：http://本机IP:3000

## 需要处理的问题

```
```text id="4m7eeg" 1. 前端 API 地址不能写死 localhost 2. WebSocket 地址要支持局域网 IP 3. 后端服务监听 0.0.0.0 4. Nginx 正确转发 WebSocket Upgrade 5. Docker 容器之间使用服务名通信 6. 数据库初始化脚本自动执行
```

```
## Nginx 配置重点
```

需要支持 WebSocket:

```
```text id="swhic8"
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";
proxy_set_header Host $host;
```

## 验收标准

```text id="1tsh8o" 1. 一条命令启动完整系统 2. 本机浏览器可以访问 3. 同一局域网手机可以访问 4. 两台设备登录不同账号可以互发消息 5. 刷新页面、断网重连后系统仍可用 6. README 中有清晰部署步骤

```
## Codex 任务
```

```
```text id="o9n0m0"
请为当前 IM 项目补充局域网部署能力。要求新增 docker-compose.yml，整合前端、Nginx、API Gateway、WebSocket Gateway、Logic、Transfer、MySQL、Redis、Kafka、Etcd 和 MinIO。Nginx 需要同时代理前端静态资源、/api HTTP 接口和 /ws WebSocket。所有服务监听 0.0.0.0，并提供 README_LAN_DEPLOY.md，说明如何在局域网内通过本机 IP 访问系统。
```

## V19：管理后台界面

### 目标

补齐企业级项目的管理端，让项目不像单纯客户端 Demo。

### 后台页面

```text id="65wfrg" 1. 管理员登录 2. 用户管理 3. 群组管理 4. 消息查询 5. 敏感词管理 6. 封禁 / 解封 7. 系统公告 8. 风控日志

```
## 核心能力
```

```
```text id="2n6ttq"
用户列表查询
用户状态修改
群组列表查询
群成员查看
消息关键词查询
敏感词增删改查
```



封禁用户  
查看风控日志

## 验收标准

```text id="qxnssm" 1. 管理员可以登录后台 2. 可以查询用户和群组 3. 可以封禁用户 4. 被封禁用户无法发消息 5. 可以配置敏感词 6. 可以查看风控日志

Codex 任务

```text id="q4mncw"

请为当前 IM 系统新增管理后台前端页面和对应接口调用。后台包括管理员登录、用户管理、群组管理、消息查询、敏感词管理、用户封禁和风控日志页面。要求复用现有风控和管理接口，如果接口不存在，请先生成接口定义文档，不要直接破坏现有业务逻辑。

## V20：监控运维展示页

### 目标

把 Prometheus / Grafana / traceId 的工程能力展示出来。

### 页面内容

可以不自己重写 Grafana，但前端可以提供一个“系统监控入口页”。

展示：

```text id="ub7e18" 当前在线人数 WebSocket 连接数 今日消息数 消息成功率 ACK 成功率 P95 延迟 Kafka 积压 服务健康状态 Grafana 链接 MinIO 链接

两种实现

轻量版

后端提供 `/api/admin/metrics/summary`：

```json id="s96atx"

```
{
 "onlineUsers": 100,
 "connections": 120,
 "messageToday": 10000,
 "successRate": 0.999,
 "ackRate": 0.995,
 "p95Latency": 35,
```

```
"kafkaLag": 0
}
```

前端展示卡片。

## 完整版

接入 Grafana iframe 或提供外链。

## 验收标准

```text id="2erp9q" 1. 后台首页能看到系统核心指标 2. 压测时指标会变化 3. 可以跳转 Grafana 4. 可以根据 traceId 查询日志

Codex 任务

```text id="gdjlxk"

请为 IM 管理后台增加系统监控展示页。要求后端提供 metrics summary 接口，前端展示在线人数、WebSocket 连接数、今日消息数、消息成功率、ACK 成功率、P95 延迟、Kafka 积压和服务健康状态，并提供 Grafana 跳转入口。

# V21：真实使用场景闭环

## 目标

让项目不只是“功能列表”，而是可以真实使用。

## 推荐真实使用场景

```text id="qru1xg" 局域网团队 IM 实验室内部通知系统 宿舍 / 小团队聊天系统 AI 任务状态通知系统 局域网文件消息系统

需要补齐的细节

```text id="v2p711"

1. 默认测试账号
2. 初始化群组
3. 系统公告
4. 文件消息
5. 群聊
6. 手机访问
7. 断线重连
8. 离线消息

## Demo 数据

初始化：

```
```text id="tqf086" admin 用户 user1 user2 user3 测试群：实验室通知群 测试群：项目讨论群
```

演示流程

```
```text id="yrvfp5"
```

1. docker compose up -d 启动系统
2. 电脑登录 user1
3. 手机登录 user2
4. user1 给 user2 发单聊消息
5. user2 断网 / 关闭页面
6. user1 继续发送消息
7. user2 重新打开页面，自动补偿离线消息
8. user1 创建群聊并发送群消息
9. 后台查看在线人数和消息 QPS
10. Grafana 查看连接数变化

## 验收标准

```
```text id="1v6dqd" 1. 可以完成完整演示流程 2. 有默认账号 3. 有初始化数据 4. 有演示说明文档 5. 局域网多设备测试通过
```

Codex 任务

```
```text id="62hdbg"
```

请为当前 IM 项目补充局域网真实使用 Demo。要求提供初始化 SQL 或 seed 脚本，创建 admin、user1、user2、user3、测试群组 and 默认好友关系；补充 DEMO\_GUIDE.md，说明如何用电脑和手机在局域网内完成登录、单聊、群聊、断线重连、离线消息补偿和后台监控演示。

## V22：桌面端 / App 壳封装

### 目标

在 Web + PWA 已完成后，再考虑封装成 App。

不要一开始做这个。

## 可选方案

### 方案一：PWA

优先级最高。

```text id="zrw92l" 优点：简单、复用 Web、手机可添加桌面 缺点：系统级能力有限

方案二：Electron 桌面端

适合 Windows / Mac 桌面 IM。

```text id="c7h422"

优点：容易封装 Web

缺点：体积大

### 方案三：Tauri 桌面端

更轻量。

```text id="arb8q5" 优点：体积小 缺点：环境配置稍复杂

方案四：Android WebView / Capacitor

适合生成 Android APK。

```text id="2cbr2i"

优点：可以像 App 一样安装

缺点：需要处理 Android 权限、网络地址配置

## 推荐路线

```text id="7ka47c" 秋招前：PWA 秋招后：Electron / Tauri 桌面端 真正需要手机 APK 时：Capacitor / Android WebView

验收标准

```text id="7s6v65"

1. PWA 可以添加桌面
2. 桌面端可以打开 IM
3. App 壳可以配置服务器地址
4. 局域网 IP 可手动配置

## Codex 任务

```text id="6paob5" 请在现有 PWA 前端基础上评估是否适合封装为 Electron 或 Tauri 桌面端。要求先输出方案对比文档，不要直接实现。重点评估打包复杂度、局域网服务器地址配置、WebSocket 支持和秋招展示价值。

V23: 项目演示、文档、压测和简历包装最终版

目标

把项目整理成真正能展示、能讲、能放简历、能给面试官看的完整项目。

必须补齐的文档

```text id="y5ziqg"

README.md

README\_LAN\_DEPLOY.md

ARCHITECTURE.md

MESSAGE\_RELIABILITY.md

GATEWAY\_ROUTING.md

OFFLINE\_SYNC.md

KAFKA\_GROUP\_DELIVERY.md

FRONTEND\_GUIDE.md

DEMO\_GUIDE.md

BENCHMARK\_REPORT.md

TROUBLESHOOTING.md

## README 首页要包含

```text id="wocnz7" 1. 项目简介 2. 技术栈 3. 架构图 4. 核心功能 5. 快速启动 6. 局域网访问方式 7. 演示账号 8. 项目截图 9. 压测结果摘要 10. 后续规划

演示截图

需要准备:

```text id="lc0l5t"

登录页

会话列表

聊天窗口

好友列表

群聊页面

手机端页面

后台用户管理

监控面板

## 简历最终表达

项目名称：

```text id="2y0uyn" 基于 Go-Zero 的企业级分布式 IM 与实时事件通知系统

项目描述：

```text id="ml18l5"

面向局域网团队通信与实时消息投递场景，设计并实现一套企业级分布式 IM 系统。系统采用 Go-Zero 微服务架构，拆分 API Gateway、Auth、User、Gateway、Logic、Transfer、Contact、Group、Conversation、Message、File、Admin 等服务，支持 WebSocket 长连接、用户认证、好友群组、单聊可靠投递、群聊异步扩散、离线消息补偿、会话列表、未读数、文件消息、管理后台、监控面板和局域网一键部署，并可扩展为 AI 产品的实时任务事件通知底座。

项目亮点：

```text id="re93cl" 1. 拆分 Gateway、Logic、Transfer 等核心服务，Gateway 负责 WebSocket 长连接维护、心跳检测和连接注册，Logic 负责消息路由、会话状态和业务处理，Transfer 负责 Kafka 异步消费和群聊扩散，实现连接层、业务层和异步投递层解耦。

1. 基于 Redis 维护 userId 到 Gateway 实例的在线路由关系，通过 TTL + 心跳续期处理异常断连，Logic 查询在线状态后通过 gRPC 调用目标 Gateway 完成实时投递，支持多 Gateway 横向扩展。
2. 设计 clientMsgId、serverMsgId、conversationSeq 和 message_delivery 投递表，结合 ACK、pending 状态和断线重连补偿机制，解决客户端重试、网络抖动和连接断开导致的消息重复与消息丢失问题。
3. 针对群聊同步扩散链路过长的问题，引入 Kafka 构建异步扩散流程，Producer 以 conversationId 作为 key 保证单会话内顺序，Transfer 消费后批量生成投递任务，并通过唯一约束保证重复消费幂等。
4. 设计 user_conversations、readSeq 和 sync_events 模型，支持会话列表、未读数、已读回执、多端同步和离线增量数据同步，提升 IM 客户端数据一致性。
5. 实现 Vue3 前端客户端和管理后台，支持登录注册、会话列表、单聊群聊、好友群组、文件消息、后台用户管理和监控面板，并通过 PWA 适配手机端访问。
6. 基于 Docker Compose 与 Nginx 实现局域网一键部署，支持电脑和手机在同一局域网内访问使用，并通过 Prometheus、Grafana 和 traceId 结构化日志观测系统运行状态。
7. 预留 task_event、agent_event 等事件消息类型，可将 AI 任务创建、Agent 执行进度、Tool Calling 结果和任务完成状态通过 WebSocket 实时推送，支持后续复用为 AI 产品事件通知底座。

Codex 任务

```text id="gm0q1x"

请为当前企业级 IM 项目补充最终展示文档和演示材料。要求生成 README.md、README\_LAN\_DEPLOY.md、ARCHITECTURE.md、FRONTEND\_GUIDE.md、DEMO\_GUIDE.md 和 BENCHMARK\_REPORT.md 的初稿，并整理项目截图清单、演示流程、默认账号、局域网部署步骤和简历项目描述。请不要修改业务代码，只补充文档。

## 五、最终完整路线总结

完整升级路径：

```text id="b6a0sq" V1-V6：后端消息投递内核 V7-V11：企业级 IM 产品能力 V12-V13：社交扩展与后台治理 V14：生产级监控和工程化 V15：AI 服务融合 V16：Web 前端客户端 V17：移动端适配与 PWA V18：局域网部署与一键启动 V19：管理后台界面 V20：监控运维展示页 V21：真实使用 Demo V22：桌面端 / App 壳封装 V23：最终文档、演示和简历包装

秋招前最推荐路线

如果时间有限，秋招前最推荐完成：

```text id="l9sqbs"

V1：可靠消息  
V2：多 Gateway 路由  
V3：ACK 与离线补偿  
V4：Kafka 群聊扩散  
V5：traceId 和压测  
V7：认证与 API Gateway  
V8：好友与群组  
V9：会话列表和未读数  
V16：基础 Web 前端  
V18：局域网部署  
V21：真实 Demo  
V23：文档和简历包装

这套完成后，项目已经可以说是：

```text id="7r6rpK" 具备真实使用闭环的企业级 IM 项目

秋招前可选加分

```text id="o38gsf"

V10：离线数据同步  
V11：文件消息

V14: Prometheus + Grafana

V17: PWA 手机端

## 秋招后再做

```text id="qyl8ad" V12: 朋友圈 V13: 完整风控后台 V15: AI Service V22: 桌面端 / Android App 壳

六、最终判断

如果只做后端，项目像：

```text id="bwpad3"

接口型后端项目

如果加上前端和局域网部署，项目会变成：

```text id="hwurg9" 可真实演示、可局域网使用、可面试讲解的完整企业级系统

最现实的目标不是一开始做原生 App，而是：

```text id="n2n5n"

Vue3 Web 前端 + PWA 手机端 + Docker Compose 局域网部署

这样你既能展示后端深度，也能展示完整系统交付能力。